EG2300 — Engineering Design II

Program Counter with T Flip-Flops

Adding Unconditional Jump

1 Context

In Tutorial 14 you built a Program Counter using T flip-flops that can count up and reset to zero. In this workshop you will add **unconditional jump** functionality — the ability to load an arbitrary 4-bit address into the PC using the asynchronous set/clear pins. You will also need to ensure that your existing reset still works correctly *alongside* the new jump logic.

2 Learning Objectives

By the end of this workshop you should be able to:

1. Use the **asynchronous set/clear** pins on T flip-flops to load an arbitrary value (jump).
2. Modify existing reset logic so that reset and jump **coexist safely** (no Set/Clear conflicts).
3. Verify that **priority** is correct: reset overrides jump, jump overrides counting.

3 T Flip-Flop Recap

A T (toggle) flip-flop has the following behaviour:

T	Clock edge	Q (next)	
0	↑	Q	(hold)
1	↑	$\overline{Q}$	(toggle)

In Logisim the T flip-flop also has two **asynchronous** pins on the bottom:

Pin label	Name	Effect (active = 1)
`1`	Preset (Set)	Forces $Q = 1$ immediately, ignores clock
`0`	Clear (Reset)	Forces $Q = 0$ immediately, ignores clock

Important

If both the Set and Clear pins are driven high at the same time, Clear wins and Q is forced to **0**. While not an error in Logisim, this is **ambiguous behaviour** on real hardware — your logic should guarantee it never happens.

Experiment: See it for yourself. Before building the full counter, place a *single* T flip-flop and explore its three bottom pins.

Figure 1: T = 1 causes Q to toggle on every rising clock edge (Q = 0 → Q = 1).

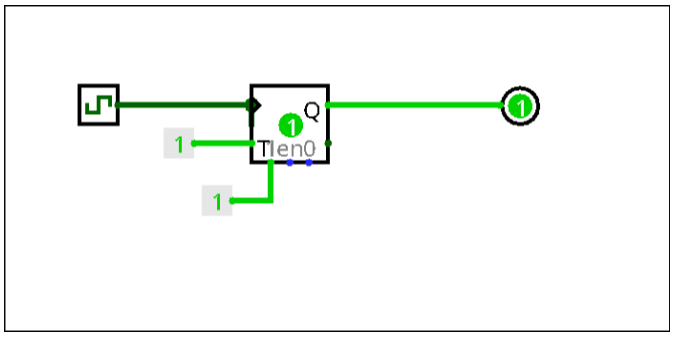

Figure 2: Async Set (pin "1") forces $Q = 1$ without a clock edge.

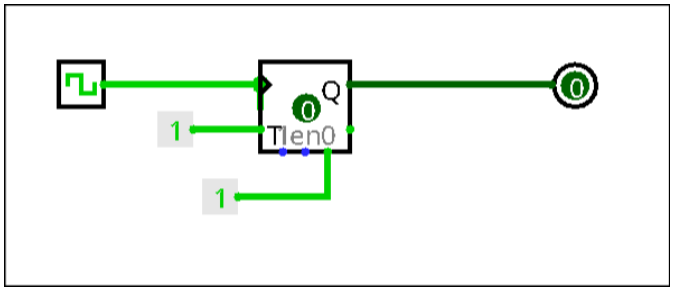

Figure 3: Async Clear (pin "0") forces $Q = 0$. Normal counting resumes when released.

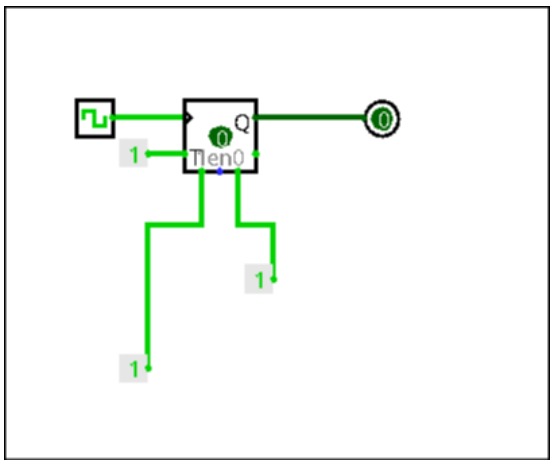

Figure 4: Both Set and Clear active — Clear wins in Logisim ($Q = 0$), but undefined on real hardware.

4 Required Behaviour

The Program Counter is a 4-bit value stored across four T flip-flops. Three operations are possible, with the following **strict priority**:

Priority	Condition	Action	Mechanism
1 (highest)	`RST` = 1	PC ← 0000	Async clear all FFs
2	`JMP_EN` = 1	PC ← `ADDR`	Async set/clear per bit
3 (default)	Neither	PC ← PC + 1	T-input toggle chain

5 Interface Specification

Create a subcircuit called `ProgramCounter_TFF` with the following pins.

Name	Width	Dir.	Meaning
`CLK`	1	in	System clock (rising-edge)
`RST`	1	in	Asynchronous reset (PC ← 0)
`JMP_EN`	1	in	Jump enable
`ADDR[3:0]`	4	in	Jump target address
`PC_OUT[3:0]`	4	out	Current program counter value

6 Tasks

6.1 Part A — Starting Point

Goal: Verify your existing Program Counter from Tutorial 14 before modifying it.
You should already have:

- Four T flip-flops wired as a ripple-carry counter (counting 0–15).
- A `RST` input that clears all flip-flops to 0 (via the async Clear pins).

Test A.

- Toggle `CLK` manually — confirm output counts 0000 → 0001 → … → 1111 → 0000.
- Assert `RST` — confirm output resets to 0000.
- If either is broken, fix it before proceeding.

6.2 Part B — Add Unconditional Jump

Goal: When `JMP_EN` is asserted, force the PC to the value on `ADDR[3:0]` using the async set/clear pins.

Currently your Clear pins are driven by `RST` alone. You will now use *both* the Set and Clear pins on each flip-flop to write an arbitrary address — while keeping reset working.

The problem. Your existing reset wires RST directly to every Clear pin. If you also wire jump logic to the Clear pins, you need to make sure:

1. Jump can drive both Set *and* Clear (to write 1s and 0s).
2. Reset still overrides jump (if both are active, reset wins).
3. Set and Clear are **never both high** on the same flip-flop.

Per-bit equations. For each bit $i \in \{0, 1, 2, 3\}$:

$$\text{SET}_i = \overline{\texttt{RST}} \texttt{ AND JMP\_EN AND ADDR}[i] \tag{1}$$

$$\text{CLR}_i = \texttt{RST OR } (\texttt{JMP\_EN AND } \overline{\texttt{ADDR}[i]}) \tag{2}$$

- If ADDR$[i] = 1$ and RST$= 0$: Set fires $\Rightarrow Q_i = 1$.
- If ADDR$[i] = 0$: Clear fires $\Rightarrow Q_i = 0$.
- If RST$= 1$: Set is forced to 0, Clear is forced to 1 $\Rightarrow Q_i = 0$ regardless of jump. **Reset wins.**
- If JMP_EN$= 0$ and RST$= 0$: both pins are 0 $\Rightarrow$ normal clocked counting.

Design Note

Verify for yourself that SET_i and CLR_i can never both be 1:
- RST $= 1 \Rightarrow \text{SET}_i = 0$ (because of $\overline{\text{RST}}$), $\text{CLR}_i = 1$. ✓
- JMP_EN $= 1$, RST $= 0$, ADDR$[i] = 1 \Rightarrow \text{SET}_i = 1$, $\text{CLR}_i = 0$. ✓
- JMP_EN $= 1$, RST $= 0$, ADDR$[i] = 0 \Rightarrow \text{SET}_i = 0$, $\text{CLR}_i = 1$. ✓
- JMP_EN $= 0$, RST $= 0 \Rightarrow \text{SET}_i = 0$, $\text{CLR}_i = 0$. ✓

No conflict in any case.

Disable counting during jump. When JMP_EN is active, the T-inputs must be forced to 0 so the clock edge does not also toggle the flip-flops. Replace the constant 1 feeding T_0:

$$T_0 = \overline{\texttt{JMP\_EN}}$$

The carry chain naturally propagates the zero, since $T_1 = \overline{\texttt{JMP\_EN}}$ AND Q_0, etc.

Design Note

Why disable T during a jump? The async pins override Q *instantly*, but the next clock edge would also toggle Q if $T = 1$. Without the NOT gate on T_0, a jump to address A could immediately become $A + 1$ on the same clock edge — defeating the purpose of the jump.

Build instructions.

1. Add a 4-bit ADDR input pin and a splitter to break it into individual bits.
2. Add a JMP_EN input pin.
3. Add shared NOT gates: $\overline{\text{RST}}$ and $\overline{\text{JMP\_EN}}$.
4. For each bit i, place:

- One NOT gate (inverts ADDR[i]).
- One 3-input AND gate for SET$_i$ (Equation 1: $\overline{\text{RST}}$, JMP_EN, ADDR[i]).
- One 2-input AND gate for the inner term (JMP_EN AND $\overline{\text{ADDR}[i]}$).
- One 2-input OR gate for CLR$_i$ (Equation 2: RST OR inner).

5. Wire SET$_i$ to the flip-flop's "1" pin.
6. **Replace** the existing direct RST → Clear wire with CLR$_i$ from the OR gate.
7. Replace the constant 1 on T_0 with $\overline{\text{JMP\_EN}}$.

Test B.

- Let the counter run to some value (e.g. 0011).
- Set ADDR = 1010 and pulse JMP_EN.
- Verify PC_OUT immediately shows 1010.
- Release JMP_EN and clock again — counter resumes from 1011.

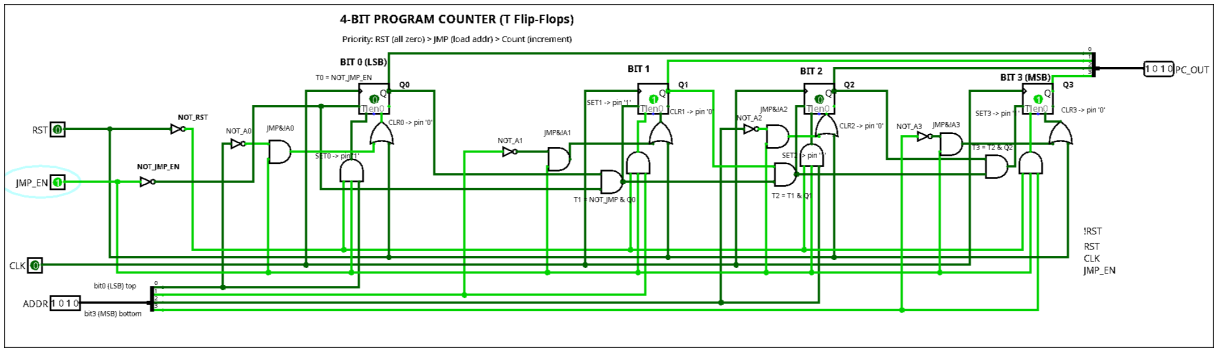

Figure 5: Jump active — ADDR = 1010, JMP_EN = 1. PC_OUT shows 1010; T-inputs are forced to 0.

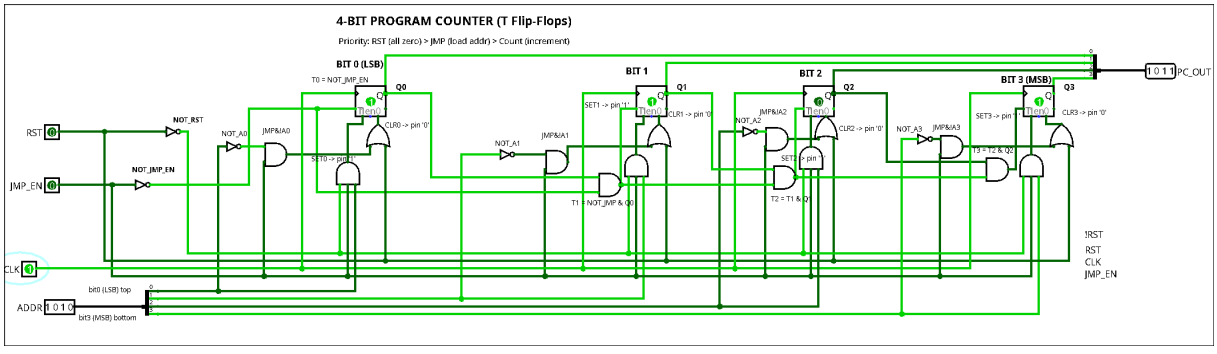

Figure 6: After releasing JMP_EN and clocking once, the counter resumes from the loaded value.

6.3 Part C — Verify Priority

Goal: Confirm that reset, jump, and counting interact correctly.

Test C — reset overrides jump.

- Run counter to 0110.
- Assert `RST`. Verify `PC_OUT` = 0000 immediately.
- While `RST` is held, also assert `JMP_EN` with `ADDR` = 1111. Verify `PC_OUT` **stays 0000** (reset wins).
- Release `RST` (keep `JMP_EN` high). Verify `PC_OUT` jumps to 1111.
- Release `JMP_EN`, clock once. Verify counter resumes: 0000.

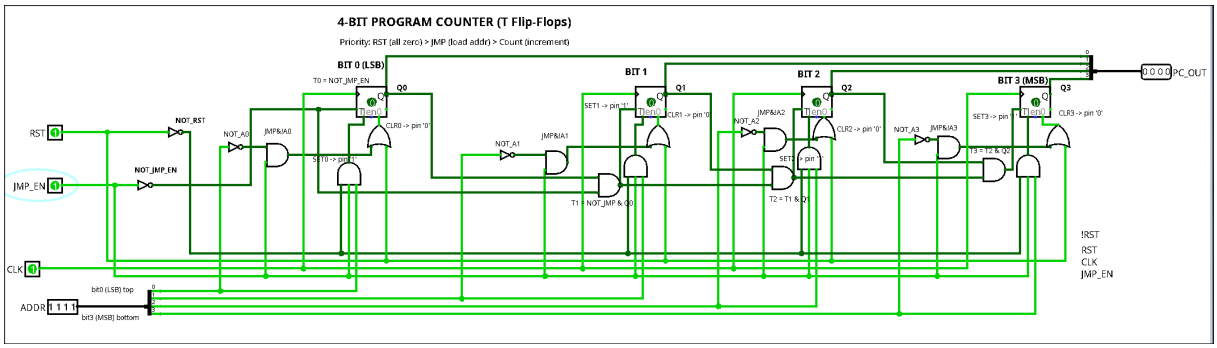

Figure 7: Reset overrides jump — `RST` = 1 and `JMP_EN` = 1 with `ADDR` = 1111, yet PC_OUT stays 0000.

7 Summary of Equations

For reference, the complete logic for each flip-flop bit i:

Signal	Equation
SET_i	$\overline{\text{RST}}$ AND JMP_EN AND ADDR$[i]$
CLR_i	RST OR (JMP_EN AND $\overline{\text{ADDR}[i]}$)
T_0	$\overline{\text{JMP\_EN}}$
T_1	$\overline{\text{JMP\_EN}}$ AND Q_0
T_2	T_1 AND Q_1 (carry chain)
T_3	T_2 AND Q_2 (carry chain)

8 What to Save

Your file must include the following subcircuit:

- `ProgramCounter_TFF`

9 Optional Extension

If you finish early:

1. **ROM integration.** Connect `PC_OUT` to your instruction ROM from Tutorial 13 and verify the PC steps through stored instructions automatically.

2. **Conditional branch.** Add external combinational logic that checks a register value and drives `JMP_EN` only when the condition is met (e.g. jump-if-zero/jump-if-not-zero [JZ/JNZ]).

3. **HALT.** Add a `HALT` input that freezes the PC (forces all T-inputs to 0 and disables set/clear).